

pcaptool Repository-Intelligence Prompt Pack

A consolidated guide to the onboarding / repo-understanding prompts used to bring Codex up to speed before code changes.

Prepared for iterative agentic work in the pcaptool repository

Purpose

This document consolidates the repository-understanding prompts and operating guidance used to onboard Codex into the pcaptool codebase before implementation work. The goal is to create durable repository intelligence rather than a one-off chat exploration.

The process is staged. Each stage produces a small number of persistent artifacts and increases Codex's understanding from package-level orientation to execution-flow, data-model, heuristic, and safe-modification awareness.

Onboarding sequence

Step	Objective	Artifacts	Mode / Skill
Step 1	Set repo guidance and exploration rules	AGENTS.md merge/update	Chat; repo guidance
Step 2	Inventory the repo and identify the main runtime path	00-repo-summary.md, 10-package-matrix.md	Chat; architect
Step 3	Map execution flows and core data models	20-execution-flows.md, 30-data-models.md	Chat; architect
Step 4	Identify heuristic families and modification hotspots	40-heuristics-catalog.md, 50-risks-hotspots.md	Chat; architect
Step 5	Convert understanding into change safety guidance	60-safe-modification-notes.md, glossary.md	Chat; architect

Working rules used throughout

- Do not modify code during repository-understanding passes.

- Inspect the repo in a controlled order: README → go.mod → entrypoint → orchestration → core packages → tests only when needed.
- Keep each pass focused on 1–2 durable artifacts.
- Prefer concrete package / file / type names over generic summaries.
- Treat heuristics, fallback logic, replay/persistence behavior, and contract-bearing outputs as high sensitivity.

Step 1 – Root repo guidance

The first step was to ensure the root AGENTS.md taught Codex how to explore the repository before coding. The important additions were: repository familiarization and exploration order, package classification model, and a definition of what “sufficient repo understanding” means.

Key additions to AGENTS.md

- Treat pcaptool as both a parser and an inference engine.
- During onboarding, identify entrypoint, runtime path, package roles, shared data structures, heuristics, and hotspots before proposing changes.
- Classify packages as entrypoint / CLI, orchestration, parsing / extraction, heuristics / inference, shared models / types, output / reporting, config / infrastructure, tests / fixtures.
- Repository understanding is incomplete until Codex can name the real entrypoint, main execution path, central structs/types, main inference logic, and riskiest modification zones.

Step 2 – Structured repo inventory

Goal: produce the first two durable architecture artifacts by identifying the true entrypoint, main runtime path, central packages, package roles, and likely hotspots.

Prompt used for Step 2

\$architect

This is a repository familiarization task for pcaptool.

Do not modify code.

Do not propose implementation changes yet unless they are strictly about missing documentation artifacts.

Goal:

Produce the first two durable architecture artifacts for this repo:

1. docs/architecture/00-repo-summary.md
2. docs/architecture/10-package-matrix.md

Context:

This repo is a CLI application that scans directories of PCAP files, extracts relevant traffic data, and builds IP-to-DNS mappings using multiple heuristics.

Payloads are often truncated, so DNS responses may be missing or unparsable, while DNS queries are often still available.

This means the repo contains both direct parsing logic and inference/correlation logic.

Use the repo-root AGENTS.md as the operating rules.

Required exploration order:

1. README and top-level docs
2. go.mod
3. main entrypoint(s): main.go and/or cmd/*
4. top-level directories
5. packages on the main runtime path
6. only then inspect high-value tests if needed

Tasks:

1. Identify the true CLI entrypoint.
2. Identify the main runtime path from CLI invocation to final outputs.
3. Identify the main packages and classify each as one of:
 - entrypoint / CLI
 - orchestration
 - parsing / extraction
 - heuristics / inference
 - shared models / types
 - output / reporting

- config / infrastructure
 - tests / fixtures
4. Identify the 5–10 most central packages only. Do not create a noisy full inventory.
 5. Identify likely hotspots or high-risk areas for future modification.
 6. Draft the two markdown artifacts.

Requirements for docs/architecture/00-repo-summary.md:

- 1–2 pages max
- repository purpose
- top-level architecture in plain language
- main runtime path
- where direct parsing ends and inference/heuristics begin
- highest-risk areas
- short “safe modification mindset” section

Requirements for docs/architecture/10-package-matrix.md:

- table or structured list
- one row per important package
- columns:
 - package
 - responsibility
 - category
 - key files
 - main inputs
 - main outputs
 - risk / notes
- include only important packages, not every package

Constraints:

- Keep outputs concise and useful
- Prefer concrete package/file names over generic descriptions
- Call out uncertainty explicitly
- Do not over-document secondary code paths
- Do not generate diagrams yet
- Do not write code
- Do not create more than these two artifacts in this step

Done when:

- you can explain the main runtime path concretely
- the two markdown artifacts are drafted
- you list any unanswered questions or ambiguities that should be resolved in the next onboarding pass

Expected artifacts

- 00-repo-summary.md — short orientation memo with purpose, runtime path, hotspot areas, and safe-modification mindset.
- 10-package-matrix.md — focused responsibility matrix for the most central packages only.

Step 3 – Execution flows and core data models

Goal: move from package awareness to control-flow and type awareness. This pass identifies three concrete execution flows, major control-flow pivots, and the most important structs/types.

Prompt used for Step 3

\$architect

This is Step 3 of repository onboarding for pcaptool.

Do not modify code.

Do not propose implementation changes yet.

Goal:

Produce the next two durable architecture artifacts:

1. docs/architecture/20-execution-flows.md
2. docs/architecture/30-data-models.md

Context:

The existing onboarding artifacts already identify:

- the true CLI entrypoint
- the main runtime path
- the central packages
- the main hotspots

Use docs/architecture/00-repo-summary.md and docs/architecture/10-package-matrix.md as prior context.

Use the repo-root AGENTS.md as the operating rules.

Tasks:

1. Refine the runtime understanding into 3 concrete execution flows:
 - main happy path
 - main heuristic / attribution path
 - main failure / fallback path
2. Identify the major control-flow pivots:
 - functions where the run meaningfully branches
 - points where parsed evidence becomes inferred attribution
 - points where optional features materially change the run
3. Identify the 10–20 most important structs/types only.
4. For each important type, determine:
 - where it is defined
 - who creates it
 - who mutates it
 - who consumes it
 - whether it is contract-bearing, intermediate, or heuristic-state
5. Explicitly identify any types or flows that look legacy, duplicated, or ambiguous.

6. Draft the two markdown artifacts.

Requirements for docs/architecture/20-execution-flows.md:

- include exactly 3 main flows:
 1. happy path
 2. heuristic / attribution path
 3. failure / fallback path
- for each flow include:
 - ordered stages
 - owning packages/functions
 - major types passed between stages
 - control-flow pivots
 - risk notes
- keep it concrete and repo-grounded
- no diagrams yet unless simple ASCII helps clarity

Requirements for docs/architecture/30-data-models.md:

- include only the most important types
- for each type include:
 - name
 - package/file
 - purpose
 - produced by
 - consumed by
 - mutated? yes/no and where
 - contract importance
 - risk / notes
- explicitly call out contract-bearing types vs intermediate working state

Constraints:

- keep outputs concise and useful
- prefer concrete function/type names over generic wording
- call out uncertainty explicitly
- do not over-document secondary paths
- do not write code
- do not create more than these two artifacts in this step

Done when:

- the 3 execution flows are clear
- the core types inventory is concrete
- major control-flow pivots are identified
- open ambiguities for the next pass are listed

Expected artifacts

- 20-execution-flows.md — exactly three flows: happy path, heuristic / attribution path, and failure / fallback path.
- 30-data-models.md — inventory of the most important contract-bearing, heuristic-state, and intermediate types.

Step 4 – Heuristics and hotspots

Goal: identify the active-path heuristic families and the most dangerous modification zones. This is the pass that converts execution understanding into policy understanding.

Prompt used for Step 4

\$architect

This is Step 4 of repository onboarding for pcaptool.

Do not modify code.

Do not propose implementation changes yet.

Goal:

Produce the next two durable architecture artifacts:

1. docs/architecture/40-heuristics-catalog.md
2. docs/architecture/50-risks-hotspots.md

Context:

The existing onboarding artifacts already identify:

- the true CLI entrypoint
- the main runtime path
- the central packages
- the three execution flows
- the key data models

Use:

- docs/architecture/00-repo-summary.md
- docs/architecture/10-package-matrix.md
- docs/architecture/20-execution-flows.md
- docs/architecture/30-data-models.md

as prior context.

Use the repo-root AGENTS.md as the operating rules.

Tasks:

1. Identify the main heuristic families in the active dnsextract path.
2. For each heuristic family, identify:
 - where it lives
 - what evidence it uses
 - what fallback behavior it implements
 - what output or contract-bearing types it can influence
 - what can go wrong if modified incorrectly
3. Distinguish:
 - direct parsing behavior
 - conservative correlation behavior

- fallback attribution behavior
- persistence/replay behavior
- identity relabeling behavior

4. Produce a prioritized hotspot list for future modifications.

5. For each hotspot, explain:

- why it is risky
- what invariants are likely to be fragile
- what tests would need to be checked before modification
- whether the risk is semantic, replay-related, output-contract-related, performance-related, or control-flow-related

Requirements for docs/architecture/40-heuristics-catalog.md:

- organize by heuristic family, not by file dump
- include only active-path heuristics
- for each heuristic include:
 - name
 - package/file/function(s)
 - purpose
 - evidence used
 - decision / fallback rule
 - downstream effect
 - risk notes
- explicitly call out precedence rules and conservative-vs-opportunistic behavior

Requirements for docs/architecture/50-risks-hotspots.md:

- prioritize the top 10 hotspots only
- for each hotspot include:
 - location
 - why it is dangerous
 - what kinds of bugs it could introduce
 - what artifacts/tests/contracts it can affect
 - safe modification notes
- include a short section:
 - "best safe entry points for future changes"
 - and
 - "areas to avoid casual modification"

Constraints:

- keep outputs concise and useful
- prefer active runtime path over legacy side paths
- call out uncertainty explicitly
- do not create more than these two artifacts in this step
- do not write code

Done when:

- the main heuristic families are explicit
- the top hotspots are prioritized
- safe vs unsafe modification zones are clear
- the next remaining ambiguities are listed

Expected artifacts

- 40-heuristics-catalog.md — active-path heuristic families grouped by behavior rather than by file.
- 50-risks-hotspots.md — prioritized top 10 hotspots, safe entry points, and areas to avoid casual modification.

Step 5 – Safe modification notes and glossary

Goal: convert all prior understanding into future-safe implementation guidance and a shared terminology set for Codex and humans.

Prompt used for Step 5

\$architect

This is Step 5 of repository onboarding for pcaptool.

Do not modify code.

Do not propose implementation changes yet.

Goal:

Produce the final two onboarding artifacts:

1. docs/architecture/60-safe-modification-notes.md
2. docs/architecture/glossary.md

Context:

The existing onboarding artifacts already identify:

- repo purpose and runtime path
- central packages
- execution flows
- key data models
- heuristic families
- risks and hotspots

Use:

- docs/architecture/00-repo-summary.md
- docs/architecture/10-package-matrix.md
- docs/architecture/20-execution-flows.md
- docs/architecture/30-data-models.md
- docs/architecture/40-heuristics-catalog.md
- docs/architecture/50-risks-hotspots.md

as prior context.

Use the repo-root AGENTS.md as the operating rules.

Tasks:

1. Convert the current architecture understanding into practical “how to modify safely” guidance.
2. Organize the safe-modification notes by change type, not by file dump.
3. For each major change type, explain:
 - where to start reading
 - what invariants are fragile
 - what artifacts or outputs can change
 - what tests should be checked first

- what mistakes to avoid
4. Include a section on:
- safest entry points for low-risk contributions
 - medium-risk changes that need targeted tests
 - high-risk zones that require explicit architecture review first
5. Produce a concise glossary of the domain concepts, data-model concepts, and repo-specific terms Codex should use consistently.

Requirements for docs/architecture/60-safe-modification-notes.md:

- practical, operational tone
- organized by change scenario, for example:
 - parser behavior changes
 - DNS-to-connection correlation changes
 - topology attribution changes
 - fallback CSV / replay changes
 - RADIUS identity changes
 - output/report formatting changes
- for each scenario include:
 - where to start
 - fragile invariants
 - likely downstream effects
 - tests to inspect first
 - safe modification advice
- include a short section:

"What to change first when debugging output drift"

Requirements for docs/architecture/glossary.md:

- concise definitions only
- include important repo terms such as:
 - DNSTransaction
 - OutputRecord
 - TopologyEntry
 - Evidence
 - fallback map / dns-ip.csv
 - active resolve
 - issuer-only fallback
 - csv+mid
 - csv+conn

- peer completion
- unresolved row
- strong DNS evidence
- contract-bearing type
- heuristic-state
- include only terms that matter for future work in this repo

Constraints:

- keep outputs concise and reusable
- prefer operational usefulness over theory
- do not create more than these two artifacts in this step
- do not write code

Done when:

- future safe modification guidance is explicit
- key terminology is standardized
- the repo onboarding set feels complete enough for future implementation tasks

Expected artifacts

- 60-safe-modification-notes.md — change-scenario driven guidance: where to start, fragile invariants, downstream effects, tests, and safe modification advice.
- glossary.md — concise definitions for the repo's key semantic terms so future prompts stay precise.

Recommended persistent output set

- docs/architecture/00-repo-summary.md
- docs/architecture/10-package-matrix.md
- docs/architecture/20-execution-flows.md
- docs/architecture/30-data-models.md
- docs/architecture/40-heuristics-catalog.md
- docs/architecture/50-risks-hotspots.md
- docs/architecture/60-safe-modification-notes.md
- docs/architecture/glossary.md

Definition of done for repository understanding

- Codex can name the real CLI entrypoint and the actual hot path.
- Codex can distinguish orchestration, parsing, heuristics, shared types, and output/reporting.

- Codex can identify the central mutable and contract-bearing types.
- Codex can describe the main heuristic families and precedence rules.
- Codex can prioritize safe vs unsafe modification zones.
- Codex can propose a small implementation plan that starts in the right place and preserves the right invariants.

Operator notes

- Use Chat mode for repository-understanding passes.
- Use the architect skill for all repo-intel passes.
- Review each preview manually before accepting generated docs.
- Keep each onboarding pass focused on one or two artifacts.
- Refresh only the affected architecture docs after major semantic or structural changes.

This prompt pack is meant to be reused for future repositories as well. Replace repo-specific names and artifact targets, but keep the staged structure.